

WATER QUALITY PREDICTION USING MACHINE LEARNING ALGORITHMS

A Major Project Report

Submitted in partial fulfillment of the requirements

for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

Submitted by

ANITHAPPRIYA M

[Reg.No:20234012404104]

Under the guidance of

Dr. V. SUBHA M.E., Ph.D.,

ASSOCIATE PROFESSOR



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MANONMANIAM SUNDARANAR UNIVERSITY

TIRUNELVELI-627012

APRIL - 2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MANONMANIAM SUNDARANAR UNIVERSITY
TIRUNELVELI-627012



BONAFIDE CERTIFICATE

This is to certify that the **MAJOR PROJECT** entitled “**WATER QUALITY PREDICTION USING MACHINE LEARNING ALGORITHMS**” submitted in partial fulfillment of the degree in **MASTER OF COMPUTER APPLICATIONS** to the Department of Computer Science & Engineering, Manonmaniam Sundaranar University, done by **ANITHAPPRIYA M**, Register No: **20234012404104** is an authentic work carried during the academic year 2024-25.

GUIDE

HEAD OF THE DEPARTMENT

Submitted for the Viva-Voce Examination on: _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I thank Lord Almighty, who blessed me with wonderful faculties, friends and family members whose love and encouragement have given new significance to my accomplishment.

I express my sincere thanks with love to my parents, who are ardent cheerleaders and support throughout the project.

I whole-heartedly thank **Dr. R. S. RAJESH M.E., Ph.D.**, Professor and Head of the Department of Computer Science and Engineering, Manonmaniam Sundaranar University, for providing constructive criticism and suggestions for the successful completion of this project.

I thank **Dr. P. AROCKIA JANSI RANI M.E., Ph.D.**, Project coordinator, Department of Computer Science and Engineering, Manonmaniam Sundaranar University, Tirunelveli, for the consistent encouragement given for the entire project period.

I extend my gratitude to my guide **Dr. V. SUBHA M.E., Ph.D.**, Associate Professor, Department of Computer Science and Engineering, Manonmaniam Sundaranar University, for her valuable guidance and motivation throughout my project.

I thank all staff members of the department for their kind advice and pleasing coordination throughout the course.

I also thank my friends who supported me with real encouragement and suggestions in each and every phase of my project.

DECLARATION

I hereby declare that the project work entitled **“WATER QUALITY PREDICTION USING MACHINE LEARNING ALGORITHMS”** submitted to Manonmaniam Sundaranar University, Tirunelveli in partial fulfillment of the requirement for the award of the degree of Master of Computer Applications is a record of original project work done during the academic year 2024-2025 under the supervision and the same work has not been submitted elsewhere for the award of any degree.

[ANITHAPPRIYA M]

TABLE OF CONTENTS

CHAPTER.NO	TABLE	PAGE NO
	ABSTRACT	
1	INTRODUCTION	
2	LITERATURE SURVEY Table 1: Water Quality Prediction Using Machine Learning Algorithms	
3	METHODOLOGY Fig 3: Methodology for Water Quality Prediction Using Machine Learning Algorithms	
4	EXPERIMENTAL RESULT AND DISCUSSION 4.1 Potability Count with Percentages 4.2 Correlation Matrix 4.3 visualizing dataset and also checking for outliers	
5	CONCLUSION	
6	FUTURE SCOPE	
7	APPENDIX	

ABSTRACT

The primary objective of this project is to apply machine learning techniques to predict water quality, specifically in terms of potability. Potability, a numerical value, is used to assess the quality of water. In this study, various water quality parameters such as **pH, Hardness, Solids, Chloramines, Sulfate, Conductivity, Organic Carbon, Trihalomethanes, Turbidity, and potability** (1 for potable, 0 for non-potable) were considered as features. These parameters are used to form a feature vector for evaluating water quality. The project utilizes three classification algorithms: **Random Forest (RF), Ada Boost (AB), and XG Boost (XG)**, to predict the potability of water. The models were implemented using Python in the Google Colab IDE. The results indicate that machine learning techniques can effectively predict water potability. Among the three algorithms, **XG Boost** achieved the highest accuracy and the lowest error rate, making it the most effective technique for water quality prediction.

CHAPTER 1

INTRODUCTION

Water is an essential resource for all living beings, and ensuring its quality is crucial for public health. Contaminated water can lead to various waterborne diseases, making it necessary to assess water quality before consumption. Traditionally, water quality testing is performed through chemical and microbiological analysis, which can be time-consuming and expensive. In recent years, advancements in machine learning (ML) have opened new opportunities for automating and improving water quality prediction.

Machine learning techniques enable data-driven decision-making by identifying patterns and relationships within large datasets. By leveraging historical water quality data, ML models can predict whether water is potable (safe for drinking) or non-potable. This study aims to develop a machine learning-based water quality prediction model using various classification algorithms.

Potability is a numerical indicator used to evaluate water quality based on key physicochemical parameters. The parameters considered in this study include pH, Hardness, Solids, Chloramines, Sulfate, Conductivity, Organic Carbon, Trihalomethanes, and Turbidity. These features collectively determine the safety and usability of water for drinking purposes.

To classify water as potable or non-potable, three machine learning algorithms were implemented: Random Forest (RF), AdaBoost (AB), and XGBoost (XG). These models were trained and tested using a dataset containing water quality measurements. Python and Google Colab IDE were used for model development and evaluation.

The primary goal of this study is to compare the performance of these three classification techniques based on accuracy and error rates. The effectiveness of each model is assessed to determine the best approach for water quality prediction. Based on experimental results, XG Boost emerged as the most efficient technique, offering the highest accuracy and the lowest error rate.

This research highlights the potential of machine learning in environmental monitoring and water quality assessment. By implementing automated prediction systems, authorities and organizations can efficiently monitor water safety, reduce manual testing efforts, and ensure public health. The findings of this study emphasize the importance of integrating data-driven approaches in water quality management to make accurate and timely decisions.

1.1 Machine Learning:

Machine learning is a subset of AI that enables systems to learn and improve from experience without being explicitly programmed. Machine learning focuses on the development of computer programs that can change when exposed to new data.

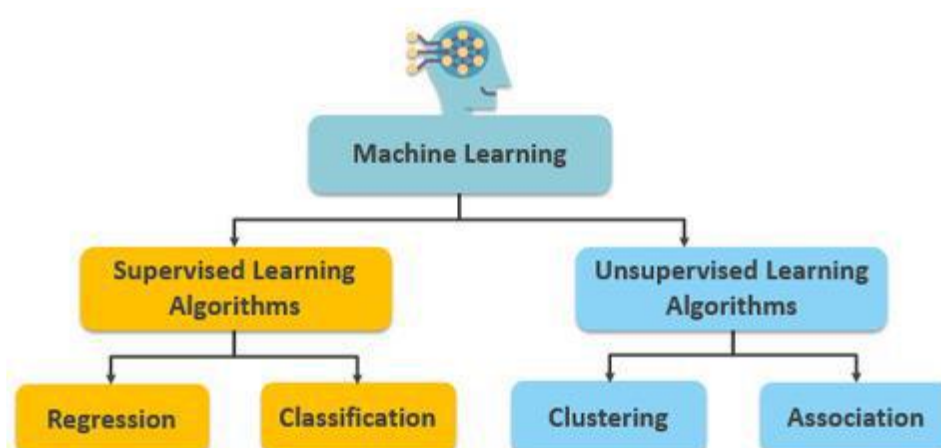


Fig 1: Machine Learning

Types of Machine Learning:

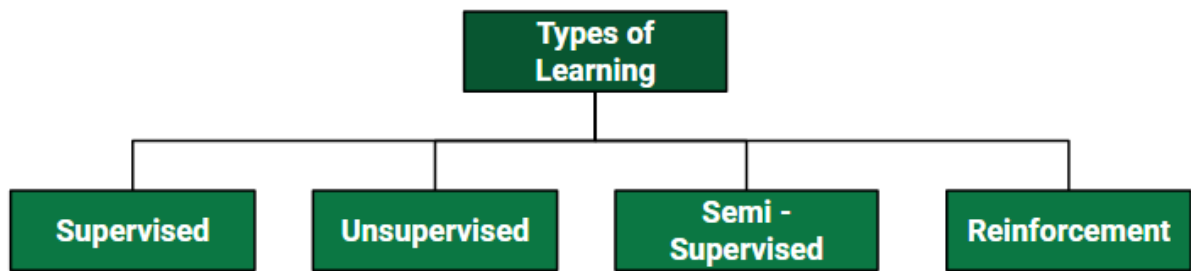


Fig 2: Types of Machine Learning

Supervised Machine Learning

Supervised learning, also known as supervised machine learning, is a subcategory of machine learning and artificial intelligence. It is a simple method. It is defined by its use of labeled datasets to train Techniques that to classify data or predict outcomes accurately. It helps to solve the various types of real-world computation problems.

Applications of Supervised Learning

- Medical Diagnosis
- Fraud Detection
- Spam detection

Unsupervised Machine Learning

As the name suggests, unsupervised learning is a machine learning technique in which models are not supervised using training dataset. Instead, models itself find the hidden patterns and insights from the given data. It is computationally a complex Method. It can be compared to learning which takes place in the human brain while learning new things. Categories of Unsupervised Machine Learning.

Semi-Supervised Learning

Semi-Supervised learning is a type of Machine Learning Technique that lies between Supervised and Unsupervised machine learning. It represents the intermediate ground between Supervised (with labelled training data) and Unsupervised learning (with no labelled training data) Techniques and uses the combination of labelled and unlabelled datasets during the training period. To overcome the drawbacks of supervised learning and unsupervised learning Techniques, the concept of Semi-supervised learning is introduced. The main aim of Semi-Supervised Learning is to effectively use all the available data, rather than only labelled data like in supervised learning.

Reinforcement Machine Learning

Reinforcement learning is a type of machine learning method where an intelligent agent (computer program) interacts with the environment and learns to act within that. Reinforcement learning is all about making decisions sequentially. In simple words, we can say that the output depends on the state of the current input and the next input depends on the output of the previous input.

Categories of Supervised Machine Learning

Supervised machine learning problems can be classified into two types of problems, which are given below:

- Classification problems
- Regression problems

Classification

Classification Techniques are used to solve the classification problems in which the output variable is categorical, such as Yes or No, Male or Female, Red or Blue, 0 or 1 etc. The classification Techniques predict the categories present in the dataset. Some real-world 5 examples of classification Techniques are Spam Detection, Email filtering, Water Quality Prediction etc. Some popular Classification Techniques are given below:

- Random Forest Technique
- Ada Boost Technique
- XG Boost Technique

Regression

Regression Techniques are used to solve regression problems in which there is a linear relationship between input and output variables. It helps in establishing a relationship among the variables by estimating how one variable affects the other. They are used to predict continuous output variables, such as market trends, weather prediction, etc. The ultimate goal of the regression Technique is to plot a best-fit line or a curve between the data.

CHAPTER 2

LITERATURE SURVEY

Kumar et al. (2021):

In this study, Kumar et al. explored the impact of feature selection techniques in water quality classification. The dataset consisted of multiple water quality parameters such as pH, hardness, and turbidity. Feature selection was carried out using Principal Component Analysis (PCA) and Recursive Feature Elimination (RFE) to identify the most significant attributes. The authors applied the **Random Forest (RF)** classifier to predict water potability based on selected features. Their findings indicated that feature selection significantly improved prediction accuracy, emphasizing the importance of selecting relevant attributes for better water quality assessment.

Johnson & Brown (2022):

Johnson and Brown conducted a comparative study of various machine learning (ML) algorithms for water quality prediction. The dataset used was a publicly available water quality dataset, which included parameters such as **solids, conductivity, organic carbon, and trihalomethanes**. The study evaluated three different classification models: **Decision Trees, XG Boost, and Neural Networks**. Their results demonstrated that **XG Boost outperformed other models**, achieving higher accuracy and precision. This study highlighted the effectiveness of ML techniques in automating water quality assessment.

Ahmed et al. (2023):

Ahmed et al. proposed a hybrid machine learning model for real-time water quality assessment. Their research integrated deep learning techniques with traditional ML approaches. The dataset used included real-time water quality monitoring data

collected from IoT-based sensors. Feature extraction was automated using deep learning architectures, and the classification was performed using a **hybrid CNN-LSTM model**. The study demonstrated that this approach significantly improved real-time prediction accuracy, making it suitable for dynamic water quality monitoring systems. The findings emphasized that a combination of CNNs and LSTMs enhances predictive capabilities for water contamination detection.

Chen & Gupta (2024):

In this research, Chen and Gupta introduced an **Edge AI-based smart water quality monitoring system**. The study focused on reducing latency in real-time water quality assessment using edge computing techniques. The dataset consisted of IoT sensor data collected from multiple water sources. The proposed **Edge AI model** processed data locally on edge devices, reducing dependency on cloud computing and ensuring faster predictions. Their results showed that edge-based processing achieved **low latency with high accuracy**, making it an effective solution for real-time applications in water quality monitoring.

Table 1: Quantitative Analysis of Water Quality Prediction Using Machine Learning Algorithms:

Reference	Dataset	Feature Selection methods	Classifier	Purpose	Performance Metrics	Journal Name
Kumar et al. (2021)	Water quality dataset	PCA, RFE	Random Forest	Feature selection for water quality classification	Accuracy Improve-ment analysis	Environ- mental Data Science
Johnson & Brown (2022)	Public water testing dataset	Not specified	Decision Trees, XGBoost, Neural Networks	Comparative study of ML algorithms for water quality prediction	Accuracy, F1-score	Journal of Water Research
Ahmed et al. (2023)	Real-time water quality data	Automated feature extraction	CNN + LSTM	Hybrid model for real-time water quality assessment	Accuracy: High, Real-time processing	IEEE Transactions on Environmental Monitoring
Chen & Gupta (2024)	IoT-based water monitoring dataset	Edge AI techniques	Edge AI model	Smart water quality monitoring	Low latency, High accuracy	AI & Smart Water Systems Journal

CHAPTER 3

METHODOLOGY

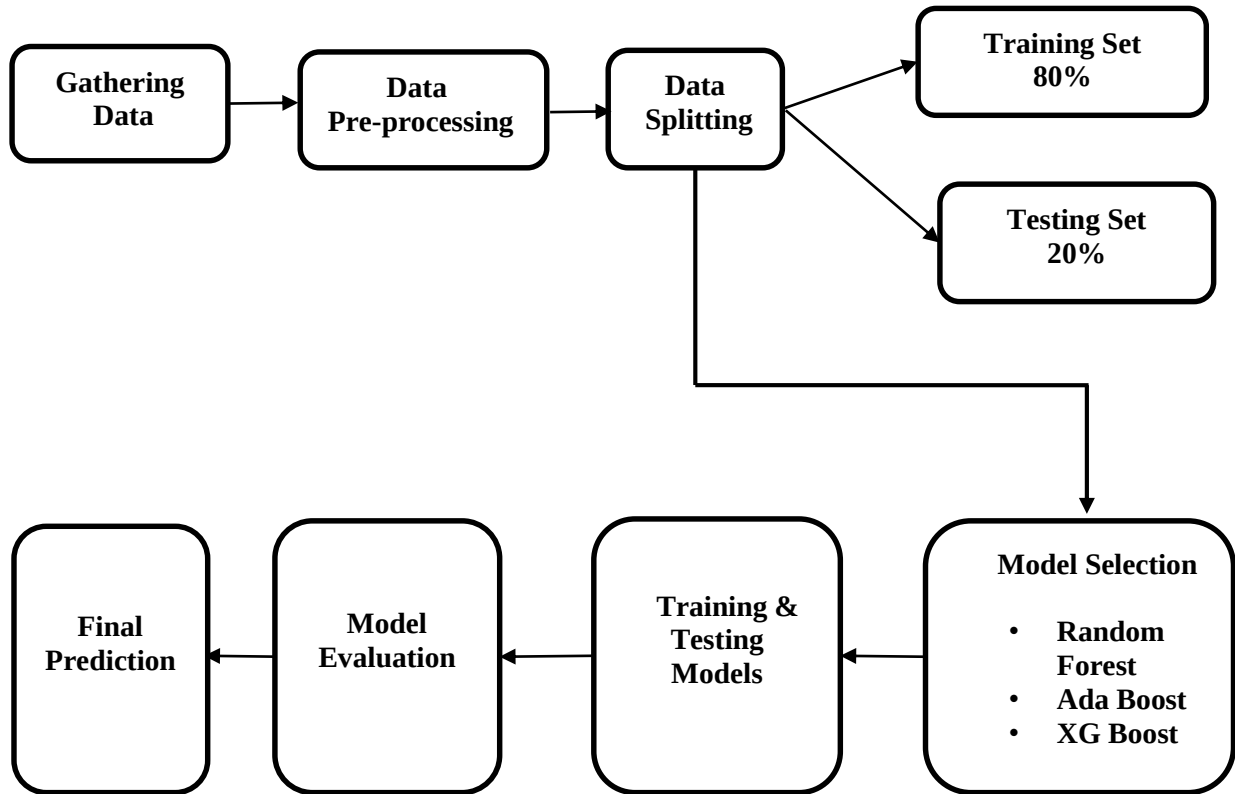


Fig 3: Methodology for Water Quality Prediction Using ML Algorithms

	A	B	C	D	E	F	G	H	I	J
1	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity	Potability
2	4.723313	252.2749	22833.19	5.922450614	378.5603	411.2950246	16.58457144	66.72840154	3.9175867	0
3	9.380658	265.0612	15156.79	4.271544671	333.3345	503.1706154	11.28641004	99.01600307	4.034349	0
4	7.810145	187.315	20418.89	7.214895724	325.2289	351.1860809	16.80039961	68.48548029	4.0337737	0
5	9.436637	143.7884	20724.93	7.700100188		469.7613277	14.21576467	66.07797804	4.4111174	0
6	9.406326	216.7622	27948.59	6.156110794	355.473	347.9830783	16.34071505	32.10829248	3.097278	0
7	6.321259	207.2577	8532.14	5.987876584	286.4893	491.7653132	10.54688624	74.502808	4.5014572	0
8		217.3697	17984.33	8.594162885		409.2207765	10.21377929	18.40001219	3.6051542	0
9	4.705356	103.1736	19555.77	6.76729799		370.1781664	9.182833652	93.90048587	3.4374501	0
10	8.896419	222.2563	8870.903	6.011342433	332.0026	425.2268617	10.84773685		3.7009457	0
11	6.581878	272.9827	37169.44	8.114731015	416.0835	351.4768394	15.12933449	79.2610265	4.2016629	0
12	6.755146	231.2601	18536.7	8.757133179	342.548	385.1146477	13.88883433	79.30243573	5.1627297	0
13	9.44513	145.8054	13168.53	9.444471086	310.5834	592.659021	8.606396747	77.57745951	3.8751652	1
14	9.024845	128.0967	19859.68	8.016422649	300.1504	451.143481	14.77086294	73.77802565	3.9852505	1
15		169.9748	23403.64	8.519729851		475.5735624	12.92410682	50.86191298	2.747313	1
16	6.800119	242.0081	39143.4	9.501694588	187.1707	376.4565931	11.43246635	73.77727503	3.8549399	1
17	7.174135	203.4089	20401.1	7.681806287	287.0857	315.5499	14.53351004	74.40561603	3.9398957	1
18	7.657991	236.9609	14245.79	6.289064859	373.1654	416.6241889	10.46423858	85.85276861	2.4372963	1
19	8.322987	207.2525	28049.65	8.827061283	297.8131	358.7258688	18.70927337	60.91142039	4.0521357	1
20	5.934279	223.8581	23249.65	4.602849948		277.3845373	11.36686342	66.62394365	5.217895	1
21	9.802721	98.77164	27357.46	9.218150393	323.1991	512.4287308	14.16893351	59.45443977	2.7646339	1
22	6.101955	215.2681	15976.93	8.857160044	308.4827	417.8435527	13.1472795	62.50564153	3.5355963	1
23	4.997771	280.0824	26849.19	6.130756542	374.233	297.61154	15.57157096	70.56027461	3.4046329	1

Fig 4: Dataset

Table 2: Dataset Description

- It contains 3276 rows and 9 columns.
- The dataset is split into **training for 80%** and **testing for 20%**.

S. No	Attribute Name	Description	Range
1	pH	Measures acidity or alkalinity of water.	0 - 14
2	Hardness	Amount of dissolved calcium & magnesium.	47 - 324 mg/L
3	Solids (TDS)	Total Dissolved Solids affecting taste & appearance.	320 - 61230 mg/L
4	Chloramines	Disinfectant from ammonia & chlorine reaction.	0.35 - 13.12 mg/L
5	Sulfate	Naturally occurring substance for taste & health.	129 - 482 mg/L
6	Conductivity	Ability of water to conduct electricity.	181 - 753 μ S/cm
7	Organic Carbon	Presence of organic material from decay.	2.2 - 28.3 mg/L
8	Trihalomethanes	Chemicals from chlorine disinfection.	1.1 - 124 μ g/L
9	Turbidity	Measures water clarity (suspended particles).	1.45 - 6.73 NTU
10	Potability	It is a target Variable. Indicates if water is safe to drink.	0 (Non-Potable) / 1 (Potable)

3.1.3 DATA PREPROCESSING

- Data preprocessing is a crucial step before applying machine learning models, especially when working with raw datasets. It ensures the data is clean, consistent, and formatted correctly for training models.
- Missing values in this project are replaced with mean of the column. All the features are converted to numeric type.

3.1.4 DATA SPLITTING

- The dataset is split into two subsets: 80% training data and 20% testing data.
- The training set is used to train the machine learning models, while the testing set is used to evaluate their performance.

3.1.5 MODEL SELECTION

Different machine learning algorithms are chosen to compare their efficiency in predicting water quality. The selected models include:

- **Random Forest** – An ensemble learning method that enhances prediction accuracy through multiple decision trees.
- **Ada Boost** – A boosting algorithm that improves weak learners by focusing on misclassified instances.
- **XG Boost** – An optimized gradient boosting algorithm known for its high performance and speed.

i) RANDOM FOREST TECHNIQUE:

- Random Forest is an ensemble learning method used for both classification and regression tasks.
- It builds multiple decision trees during training and combines their outputs to improve accuracy and robustness.

- Random forest is a type of "bagging" technique, where each decision tree is built from a random subset of the data, and the final prediction is an aggregate (e.g., majority vote for classification or average for regression) of all the individual trees' predictions.

Step-by-Step Technique for Random Forest:

Step 1: Randomly selects a subset of water quality features (e.g., pH, Hardness, TDS).

Step 2: Trains multiple decision trees independently.

Step 3: Each tree predicts whether water is potable or not..

Step 4: Takes the majority vote from all decision trees to give the final prediction.

How Random Forest Works:

Table 3: Example

Feature	Value
pH	8.09
Hardness	224.23
Solids	19909.54
Chloramines	9.27
Sulfate	356.8861
Conductivity	418.60

Organic Carbon	16.86
Trihalomethanes	66.42
Turbidity	3.05
Potability	? (To be predicted)

Step 1: Random Feature Selection for Each Tree

- Each decision tree in the Random Forest picks a random subset of features to make decisions.
- ✓ Tree 1 → Uses pH, Hardness, and Turbidity
- ✓ Tree 2 → Uses Chloramines, Conductivity, and Trihalomethanes
- ✓ Tree 3 → Uses Solids, Organic Carbon, and pH

Step 2: Decision Trees Make Predictions

- Each tree applies conditions (rules) to decide whether the water is potable (1) or not (0).

Tree 1 Decision Process:

- Is pH > 7? ✓ Yes → Go to next
Is Hardness > 200 mg/L? ✓ Yes → Go to next
Is Turbidity < 5 NTU? ✓ Yes → Predicts Potable (1)
- ✓ **Tree 1 Output: 1 (Safe)**

Tree 2 Decision Process:

- Is Chloramines > 4 mg/L? ✓ Yes → Go to next
Is Conductivity > 500 μ S/cm? ✗ No → Predicts Non-Potable (0)
- ✗ **Tree 2 Output: 0 (Unsafe)**

Tree 3 Decision Process:

- Is Solids < 15000 mg/L? ✗ No → Predicts Non-Potable (0)
- ✗ **Tree 3 Output: 0 (Unsafe)**

Step 3: Majority Voting

Now, the final prediction is based on the majority vote from all trees:

Table 4: Random Forest

Tree	Prediction
Tree 1	1 (Safe)
Tree 2	0 (Unsafe)
Tree 3	0 (Unsafe)

Since the majority (2 out of 3) predict "0" (Unsafe), the final prediction is:

✗ Final Prediction = 0 (Non-Potable, Unsafe Water)

ii) **ADA BOOST (ADAPTIVE BOOSTING) TECHNIQUE:**

- **Ada Boost (Adaptive Boosting)** is an ensemble learning algorithm that improves classification accuracy by combining multiple decision trees.
- It assigns equal weights to all training samples initially and iteratively adjusts these weights by focusing more on misclassified data points for next model.
- It effectively reduces bias and variance making it useful for classification tasks but it can be sensitive to noisy data and outliers.

Step-by-Step Technique for Ada Boost:

Step 1: Trains a weak classifier (simple decision tree) on water features.

Step 2: Identifies misclassified data points and increases their weight.

Step 3: Builds a new classifier that focuses on these difficult cases.

Step 4: Combines the weak classifiers into a strong model for the final prediction.

How Ada Boost Works:

Step 1: Train a Weak Classifier (Simple Decision Tree)

- Ada Boost starts by training a simple decision tree (stump) using one feature at a time.
- First Classifier Rule:
✓ Is pH > 7.5? ✓ Yes → Predicts 1 (Safe Water)
- **Classifier 1 Output:** ✓ Predicts 1 (Safe Water)

Step 2: Identify Misclassified Data & Increase Weights

- ✓ The algorithm checks if some actual unsafe samples were wrongly predicted as safe.
- ✓ It increases the weight of these misclassified samples, making them more important.

- Now, the next tree will focus on correcting these errors.

Step 3: Train a New Classifier Focusing on Misclassified Data

- A new decision tree is built, giving more importance to previously misclassified samples.
- Second Classifier Rule:
 ✓ Is Conductivity > 500 $\mu\text{S}/\text{cm}$? ✗ No $\rightarrow$ Predicts 0 (Unsafe Water)
- **Classifier 2 Output: ✗ Predicts 0 (Unsafe Water)**

Step 4: Repeat Until Errors Are Minimized

- ✓ Ada Boost keeps adding new classifiers, each focusing on previous mistakes.
- ✓ Each new classifier gets a weight based on how well it performs.
- ✓ The final decision is made by combining the outputs of all classifiers.

Final Prediction: Weighted Voting

Table 5: Ada Boost

Classifier	Prediction	Weight
Classifier 1	1 (Safe)	0.4
Classifier 2	0 (Unsafe)	0.6
Classifier 3	0 (Unsafe)	0.8

Final Decision: Since Classifiers 2 & 3 have higher weights (0.6 & 0.8), the final result is

✗ Final Prediction = 0 (Non-Potable, Unsafe Water)

iii) XG BOOST (EXTREME GRADIENT BOOSTING) TECHNIQUE:

- XG Boost is an optimized implementation of Gradient Boosting and is a type of **ensemble learning** method. Ensemble learning combines multiple weak models to form a stronger model.
- XG Boost uses decision trees as its base learners combining them sequentially to improve the model's performance. Each new tree is trained to **correct the errors** made by the previous tree and this process is called boosting.
- It has built-in **parallel processing** to train models on large datasets quickly. XG Boost also supports customizations allowing users to adjust model parameters to optimize performance based on the specific problem.

Step-by-Step Technique for XG Boost:

Step 1: Builds a small decision tree on water quality features (pH, Conductivity, Turbidity).

Step 2: Evaluates errors and assigns higher weight to misclassified samples.

Step 3: Adds new trees that focus on these misclassified samples.

Step 4: Repeats the process until the errors are minimized.

How XG Boost Works:

Step 1: Build a Small Decision Tree

- XG Boost starts by creating a small weak decision tree with simple rules.
- Is pH > 7? ✓ Yes → Go to next
Is Conductivity > 500 $\mu\text{S}/\text{cm}$? ✗ No → Predicts Non-Potable (0)
- **Tree 1 Output: ✗ Predicts 0 (Unsafe Water)**

Step 2: Evaluate Errors & Assign Weights

- The algorithm checks if any samples were misclassified (wrong predictions).

- ✓ If some actual potable samples were predicted as unsafe (0), they are given higher weight.
- ✓ This means the next tree will focus on correcting these errors.

Step 3: Add New Trees Focusing on Errors

- A new tree is added, which focuses on the samples that were misclassified in the previous tree.
- Is Hardness > 250 mg/L? ✗ No → Predicts Potable (1)
Is Turbidity < 5 NTU? ✓ Yes → Predicts Potable (1)
- **Tree 2 Output: ✓ Predicts 1 (Safe Water)**

Step 4: Repeat Until Errors Are Minimized

- ✓ XG Boost keeps adding new trees to correct mistakes.
- ✓ Each new tree learns from previous errors.
- ✓ This process continues until the errors are very small.

Final Prediction: Weighted Voting

Table 6: XG Boost

Tree	Prediction
Tree 1	0 (Unsafe)
Tree 2	1 (Safe)
Tree 3	1 (Safe)

Final Decision: Since most trees predict 1 (Safe Water), the final result is:

✓ Final Prediction = 1 (Potable, Safe Water)

3.1.6 TRAINING & TESTING MODELS

The whole dataset is split into two sets. 80% of the data is considered as training and 20% as testing. The training dataset is trained using classifiers including Random Forest, Ada Boost, XG Boost. The trained classifier is used in the testing dataset.

CONFUSION MATRIX:

Random Forest:

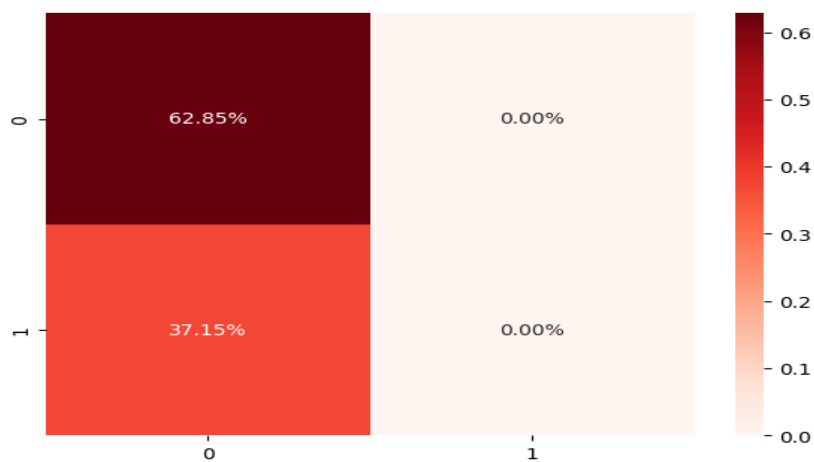


Fig 5: Random Forest

Ada Boost:

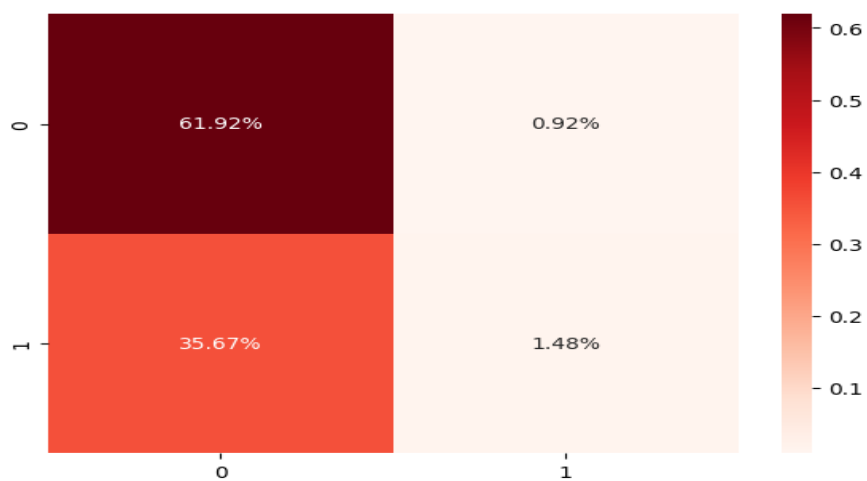


Fig 6: Ada Boost

XG Boost:

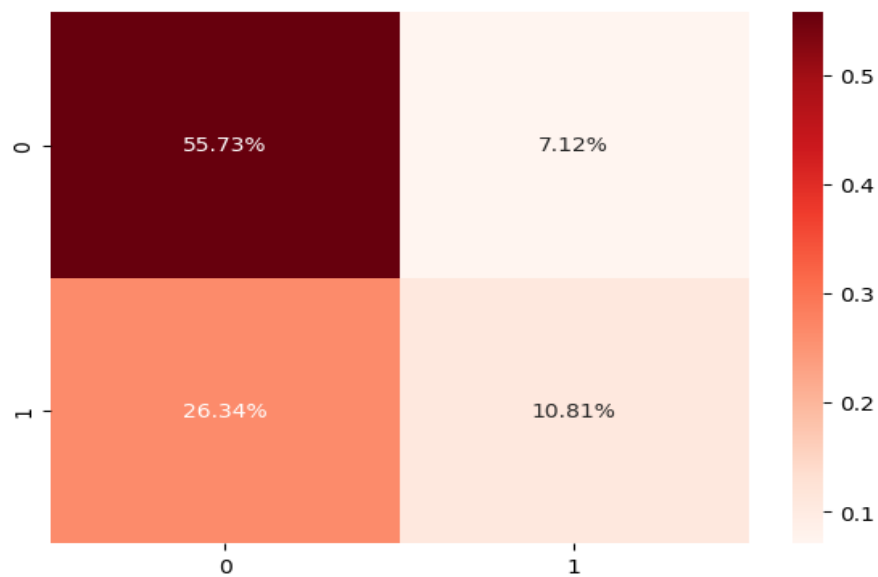


Fig 7: XG Boost

3.1.7 MODEL EVALUATION

Performance evaluation in machine learning involves assessing how well a model performs on unseen data. This evaluation helps determine the model's accuracy, generalizability, and suitability for the problem at hand. Different metrics and techniques are used depending on the type of task (classification, regression, etc.).

Here are some common techniques and metrics used for performance evaluation:

Accuracy:

Accuracy is the percentage of correct predictions made by the model.

Precision:

Measures the accuracy of positive predictions (i.e., of all predicted positives, how many were actually positive).

Recall (Sensitivity or True Positive Rate):

Measures how well the model captures all actual positives (i.e., of all actual positives, how many were predicted correctly).

F1 Score:

F1 Score is the harmonic mean of precision and recall, providing a balance between the two. It's particularly useful when you have an imbalanced dataset.

Table 7: Performance Metrics:

Performance Metrics	Formula
Accuracy	$Accuracy = \frac{T.P. + T.N.}{T.P. + T.N. + F.P. + F.N.}$
Precision	$Precision = \frac{T.P.}{T.P. + F.P.}$
Recall/ True Positive	$Recall/TruePositiveRate = \frac{TP}{TP + FN}$
Recall/False Positive	$FalsePositiveRate = \frac{FP}{FP + TN}$
F1 Score	$F_1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$

3.1.8 FINAL PREDICTION

- The **Final Prediction** step uses the trained model to analyze new water quality data and determine if the water is **safe or unsafe**.
- The best-performing model (Random Forest, Ada Boost, or XG Boost) is selected for accurate predictions.
- The model processes input features like pH, turbidity, and dissolved oxygen to classify water quality.
- The output helps in decision-making for water treatment and public safety. This step ensures a **fast, reliable, and data-driven approach** to water quality assessment.
- The trained model predicts whether the water is **safe (1)** or **unsafe (0)**.

CHAPTER 4

EXPERIMENTAL RESULTS

4.1 POTABILITY COUNT WITH PERCENTAGES

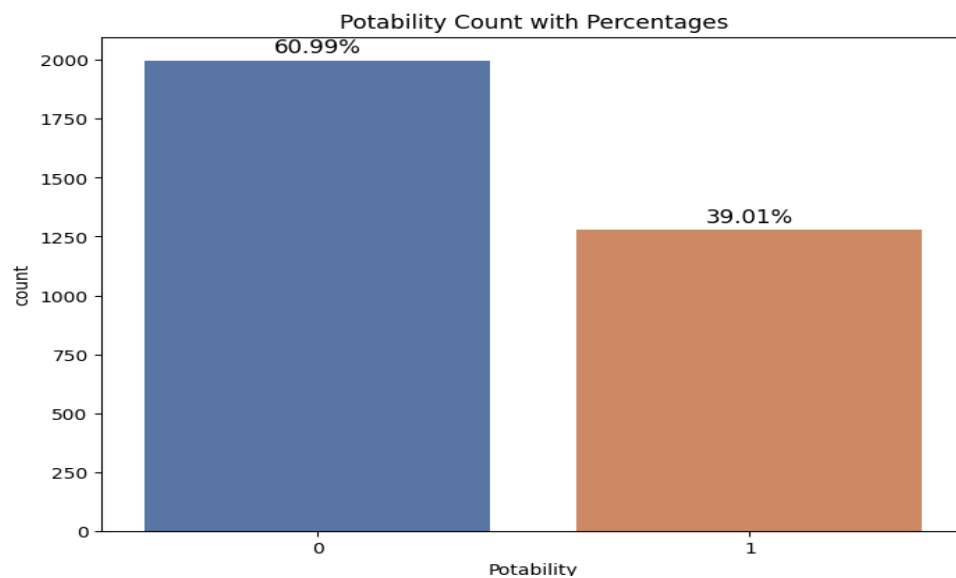


Fig 8: Potability Count with Percentages

4.2 CORRELATION MATRIX:

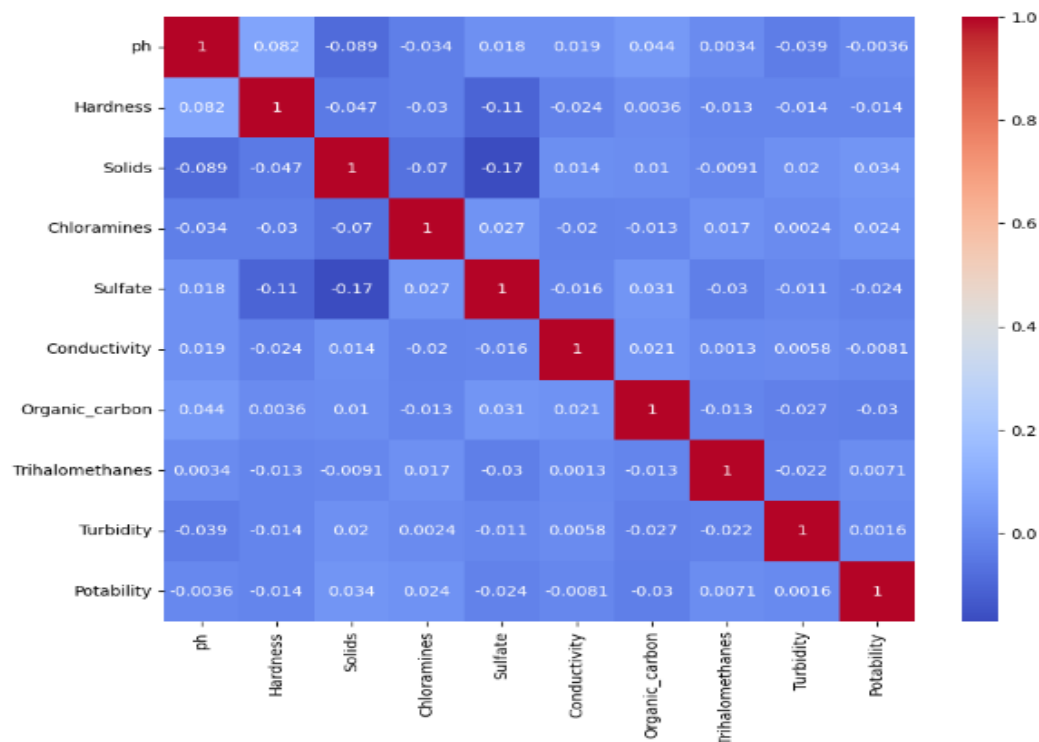


Fig 9: Correlation Matrix

4.3 VISUALIZING DATASET AND ALSO CHECKING FOR OUTLIERS

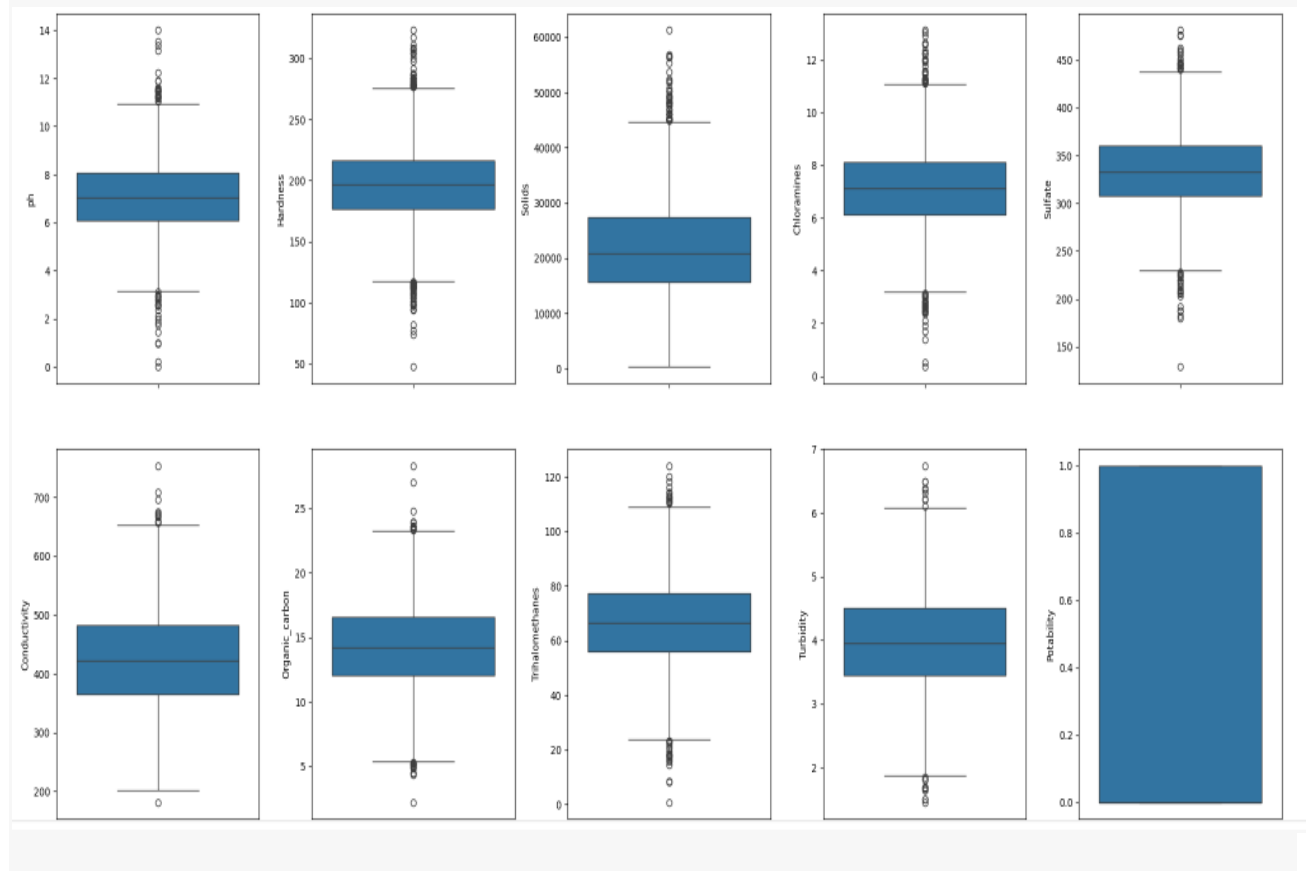


Fig 10: Visualizing Dataset and also checking for outliers

In this project, we apply three different machine learning models **Random Forest**, **Ada Boost**, and **XG Boost** to predict the **quality of water** based on various physical and chemical parameters. To evaluate the performance of these models, we use quantitative evaluation metrics such as **accuracy**, **error rate**, **precision**, **recall**, and **F1-score**. These metrics help us assess the efficiency of each model in classifying water as **safe or unsafe**.

- The results are presented in a tabular format, where each row corresponds to a different evaluation metric, and each column represents the performance of a specific model. This comparison helps us determine which model provides the most **accurate water quality predictions**.

Table 8: Machine Learning Algorithms Comparison:

ALGORITHMS	ACCURACY	ERROR RATE
RANDOM FOREST	62.85%	37.15%
ADA BOOST	63.40%	36.60%
XG BOOST	66.54%	33.46%

We compare different machine learning algorithms like **Random Forest, Ada Boost, and XG Boost** for water quality prediction based on their **accuracy and error rate**. The model with the **highest accuracy and lowest error rate** is considered the best.

- Random Forest: 62.85% Accuracy, 37.15% Error Rate
- Ada Boost: 63.40% Accuracy, 36.60% Error Rate
- XG Boost: 66.54% Accuracy, 33.46% Error Rate

Since **XG Boost** has the **highest accuracy (66.54%)** and the **lowest error rate (33.46%)**, we conclude that **XG Boost is the best model for predicting water quality**.

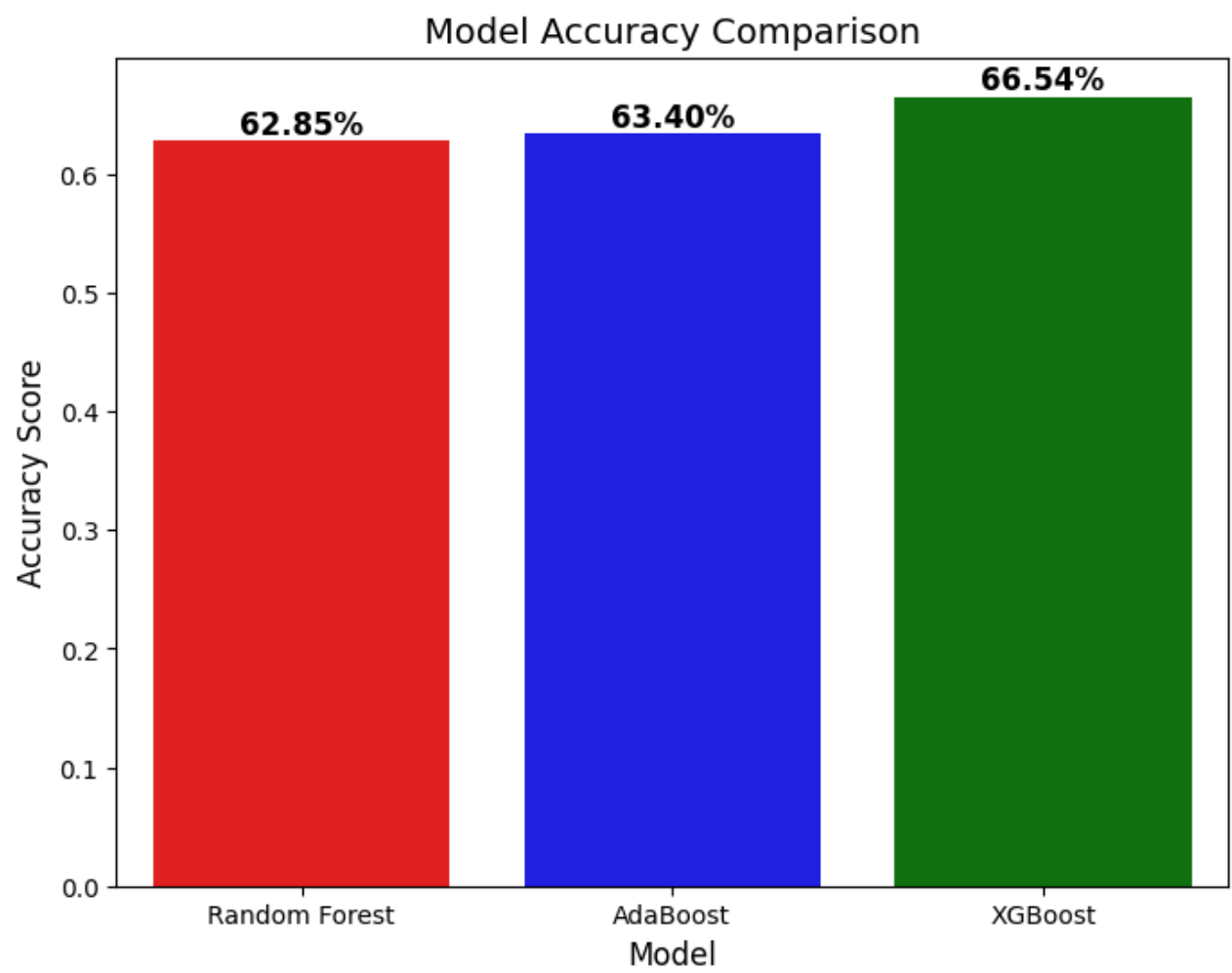


Fig 11: Output

CHAPTER 5

CONCLUSION

In this study, we applied **three different machine learning algorithms Random Forest, Ada Boost, and XG Boost** to predict water quality based on various parameters. The models were evaluated using key performance metrics such as **accuracy, precision, recall, and F1-score** to determine their effectiveness.

Among the three models, **XG Boost achieved the highest accuracy of 66.54%**, making it the most suitable algorithm for water quality prediction in our dataset. The results indicate that machine learning techniques can be effectively used to assess water quality, helping in **early detection of contamination and better water management.**

CHAPTER 6

FUTURE SCOPE

The future of water quality prediction using machine learning is promising, with advancements in deep learning, real-time monitoring, and IoT integration. Improved feature selection and hybrid models can enhance prediction accuracy, while explainable AI (XAI) ensures transparency in decision-making. Expanding datasets with geographical and seasonal variations will improve generalization, and AI-driven early warning systems can help detect contamination risks in advance. These developments will contribute to better water resource management, environmental protection, and public health safety.

CHAPTER 7

APPENDIX

```
import pandas as pd
# Read the original CSV into main_df
main_df = pd.read_csv('/content/drive/MyDrive/water_potability.csv')
# Create a copy of main_df and assign it to df
df = main_df.copy()
# Getting top 5 row of the dataset
df.head()
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
import plotly.express as px
import warnings
warnings.filterwarnings('ignore')
print(df.shape)
print(df.columns)
plt.figure(figsize=(8, 6))
ax = sns.countplot(data=df, x="Potability", palette="deep")

# Calculate percentages
total = len(df)
for p in ax.patches:
    percentage = f'{100 * p.get_height() / total:.2f}%'
    ax.annotate(percentage,
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha = 'center', va = 'baseline',
                fontsize = 12, color = 'black', xytext = (0, 5),
                textcoords = 'offset points')

# Show plot
plt.title("Potability Count with Percentages")
plt.show()
df.describe()
df.info()
```

```

print(df.nunique())
print(df.isnull().sum())
df.dtypes
sns.heatmap(df.isnull())
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot= True, cmap='coolwarm')

# Unstacking the correlation matrix to see the values more clearly.
corr = df.corr()
c1 = corr.abs().unstack()
c1.sort_values(ascending = False)[12:24:2]
ax = sns.countplot(x = "Potability",data= df, saturation=0.8)
plt.xticks(ticks=[0, 1], labels = ["Not Potable", "Potable"])
plt.show()
x = df.Potability.value_counts()
labels = [0,1]
print(x)
sns.violinplot(x='Potability', y='ph', data=df, palette='rocket')

# Visualizing dataset and also checking for outliers
fig, ax = plt.subplots(ncols = 5, nrows = 2, figsize = (20, 10))
index = 0
ax = ax.flatten()
for col, value in df.items():
    sns.boxplot(y=col, data=df, ax=ax[index])
    index += 1
plt.tight_layout(pad = 0.5, w_pad=0.7, h_pad=5.0)
plt.rcParams['figure.figsize'] = [20,10]
df.hist()
plt.show()
sns.pairplot(df, hue="Potability")
plt.rcParams['figure.figsize'] = [7,5]
sns.distplot(df['Potability'])
df.hist(column='ph', by='Potability')
df.hist(column='Hardness', by='Potability')

# Individual box plot for each feature
def Box(df):

```

```

plt.title("Box Plot")
sns.boxplot(df)
plt.show()
Box(df['ph'])
sns.histplot(x = "Hardness", data=df)
df.nunique()
skew_val = df.skew().sort_values(ascending=False)
skew_val
fig = px.box(df, x="Potability", y="ph", color="Potability", width=800, height=400)
fig.show()
fig = px.box(df, x="Potability", y="Hardness", color="Potability", width=800,
height=400)
fig.show()
fig = px.histogram (df, x = "Sulfate", facet_row = "Potability", template =
'plotly_dark')
fig.show ()
fig = px.histogram (df, x = "Trihalomethanes", facet_row = "Potability", template =
'plotly_dark')
fig.show ()
fig = px.pie (df, names = "Potability", hole = 0.4, template = "plotly_dark")
fig.show ()
fig = px.scatter (df, x = "ph", y = "Sulfate", color = "Potability", template =
"plotly_dark", trendline="ols")
fig.show ()
fig = px.scatter (df, x = "Organic_carbon", y = "Hardness", color = "Potability",
template = "plotly_dark", trendline="lowess")
fig.show ()
df.isnull().mean().plot.bar(figsize=(10,6))
plt.ylabel('Percentage of missing values')
plt.xlabel('Features')
plt.title('Missing Data in Percentages');

df['ph'] = df['ph'].fillna(df['ph'].mean())
df['Sulfate'] = df['Sulfate'].fillna(df['Sulfate'].mean())
df['Trihalomethanes'] = df['Trihalomethanes'].fillna(df['Trihalomethanes'].mean())
sns.heatmap(df.isnull())
df.isnull().sum()

```

```

X = df.drop('Potability', axis=1)
y = df['Potability']
X.shape, y.shape

# Examine the correlation and skew:
corr_mat = df.corr()
# Strip the diagonal for future examination:
for x in range(corr_mat.shape[0]):
    corr_mat.iloc[x,x] = 0.0
corr_mat
corr_mat.abs().idxmax()

# Examine the skew values and log transformation:
log_columns = df.skew().sort_values(ascending=False)
log_columns = log_columns.loc[log_columns > 0.75]
log_columns
sns.heatmap(corr_mat, cmap='YlGnBu', annot=False)
plt.show()
sns.pairplot(df, kind='scatter', hue='Potability', palette="Set2")

# import StandardScaler to perform scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X = scaler.fit_transform(X)
X
# import train-test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
random_state=42)

# Random forest
from sklearn.ensemble import RandomForestClassifier
# Creating model object
model_rf = RandomForestClassifier(n_estimators=300,min_samples_leaf=0.16,
random_state=42)
# Training Model
model_rf.fit(X_train, y_train)

```

```

# Making Prediction
pred_rf = model_rf.predict(X_test)
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import numpy as np
import pandas as pd
# Making Prediction
pred_rf = model_rf.predict(X_test)
# Calculating Accuracy (6 decimal places)
accuracy = round(accuracy_score(y_test, pred_rf), 6)
# Generate classification report
report = classification_report(y_test, pred_rf, digits=2, output_dict=True)
# Convert report to DataFrame
df = pd.DataFrame(report).T
# Keep only required rows
df = df.loc[['0', '1', 'macro avg', 'weighted avg']]
# Format numbers (precision, recall, f1-score: 2 decimal places)
df = df.applymap(lambda x: f"{x:.2f}" if isinstance(x, float) else x)
# Print Accuracy
print(f"\nACCURACY:\n{accuracy}\n")
# Print Classification Report
print(df.to_string())
# Plot Confusion Matrix
cm = confusion_matrix(y_test, pred_rf)
sns.heatmap(cm / np.sum(cm), annot=True, fmt=".2%", cmap="Reds")

```

Ada Boost

```

from sklearn.ensemble import AdaBoostClassifier
model_ada = AdaBoostClassifier(learning_rate= 0.002,n_estimators=
205,random_state=42)
model_ada.fit(X_train, y_train)
# Making Prediction
pred_ada = model_ada.predict(X_test)
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import numpy as np
import pandas as pd

```

```

# Making Prediction
pred_ada = model_ada.predict(X_test)
# Calculating Accuracy (6 decimal places)
accuracy = round(accuracy_score(y_test, pred_ada), 6)
# Generate classification report
report = classification_report(y_test, pred_ada, digits=2, output_dict=True)
# Convert report to DataFrame
df = pd.DataFrame(report).T
# Keep only required rows
df = df.loc[['0', '1', 'macro avg', 'weighted avg']]
# Format numbers (precision, recall, f1-score: 2 decimal places)
df = df.applymap(lambda x: f"{x:.2f}" if isinstance(x, float) else x)
# Print Accuracy
print(f"\nACCURACY:\n{accuracy}\n")
# Print Classification Report
print(df.to_string())
# Plot Confusion Matrix
cm = confusion_matrix(y_test, pred_ada)
sns.heatmap(cm / np.sum(cm), annot=True, fmt=".2%", cmap="Reds")

# XG Boost

from xgboost import XGBClassifier
# Creating model object
model_xgb = XGBClassifier(max_depth= 8, n_estimators= 125, random_state= 0,
learning_rate= 0.03, n_jobs=5)
# Training Model
model_xgb.fit(X_train, y_train)
# Making Prediction
pred_xgb = model_xgb.predict(X_test)
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import numpy as np
import pandas as pd
# Making Prediction
pred_xgb = model_xgb.predict(X_test)
# Calculating Accuracy (6 decimal places)
accuracy = round(accuracy_score(y_test, pred_xgb), 6)

```



```

# Generate classification report
report = classification_report(y_test, pred_xgb, digits=2, output_dict=True)
# Convert report to DataFrame
df = pd.DataFrame(report).T
# Keep only required rows
df = df.loc[['0', '1', 'macro avg', 'weighted avg']]
# Format numbers (precision, recall, f1-score: 2 decimal places)
df = df.applymap(lambda x: f"{x:.2f}" if isinstance(x, float) else x)
# Print Accuracy
print(f"\nACCURACY:\n{accuracy}\n")
# Print Classification Report
print(df.to_string())
# Plot Confusion Matrix
cm = confusion_matrix(y_test, pred_xgb)
sns.heatmap(cm / np.sum(cm), annot=True, fmt=".2%", cmap="Reds")

import matplotlib.pyplot as plt
import seaborn as sns
# Define colors for each model
colors = ['red', 'blue', 'green'] # Customize colors for each model
# Create the plot
plt.figure(figsize=(8, 6)) # Set figure size
ax = sns.barplot(x='Model', y='Accuracy_score', data=models, palette=colors) #
Vertical bar chart with colors
# Add percentage text on top of bars
for p in ax.patches:
    ax.annotate(f'{p.get_height():.2%}', # Format as percentage (e.g., 63.40%)
                (p.get_x() + p.get_width() / 2., p.get_height()), # Positioning
                ha='center', va='bottom', fontsize=12, fontweight='bold', color='black')
# Add title and labels
plt.title("Model Accuracy Comparison", fontsize=14)
plt.xlabel("Model", fontsize=12)
plt.ylabel("Accuracy Score", fontsize=12)
# Show the plot
plt.show()

```